POSIX

RTOS

- As the complexities in the embedded applications increase, use of an operating system brings in lot of advantages.
- Most embedded systems also have real-time requirements demanding the use of Real time Operating Systems (RTOS) capable of meeting the embedded system requirements.
- Real-time Operating System allows realtime applications to be designed and expanded easily.
- The use of an RTOS simplifies the design process by splitting the application code into separate tasks.
- An RTOS allows one to make better use of the system recourses by providing with valuable services such as semaphores, mailboxes, queues, time delays, time outs…etc.

RTOS

- One major subclass of embedded systems is real-time embedded systems.
- A real time system is one that has timing constraints.
- Real-time system’s performance is specified in terms of ability to make calculations or decisions in a timely manner.
- These important calculations have deadlines for completion.
- For example if the real-time system is a part of an airplane’s flight control system, single missed deadline is sufficient to endanger the lives of the passengers and crew.

Basic functions of OS

- Process management
- Memory management
- Interrupt handling
- Exception handling
- Process Synchronization (IPC)
- Process scheduling
- Disk management

RTOS

- “A good RTOS is one that has a bounded (predictable) behavior under all system load scenario i.e. even under simultaneous interrupts and thread execution
- RTOS occupy little space from 10 KB to 100KB as compared to the General Operating systems which take hundreds of megabytes

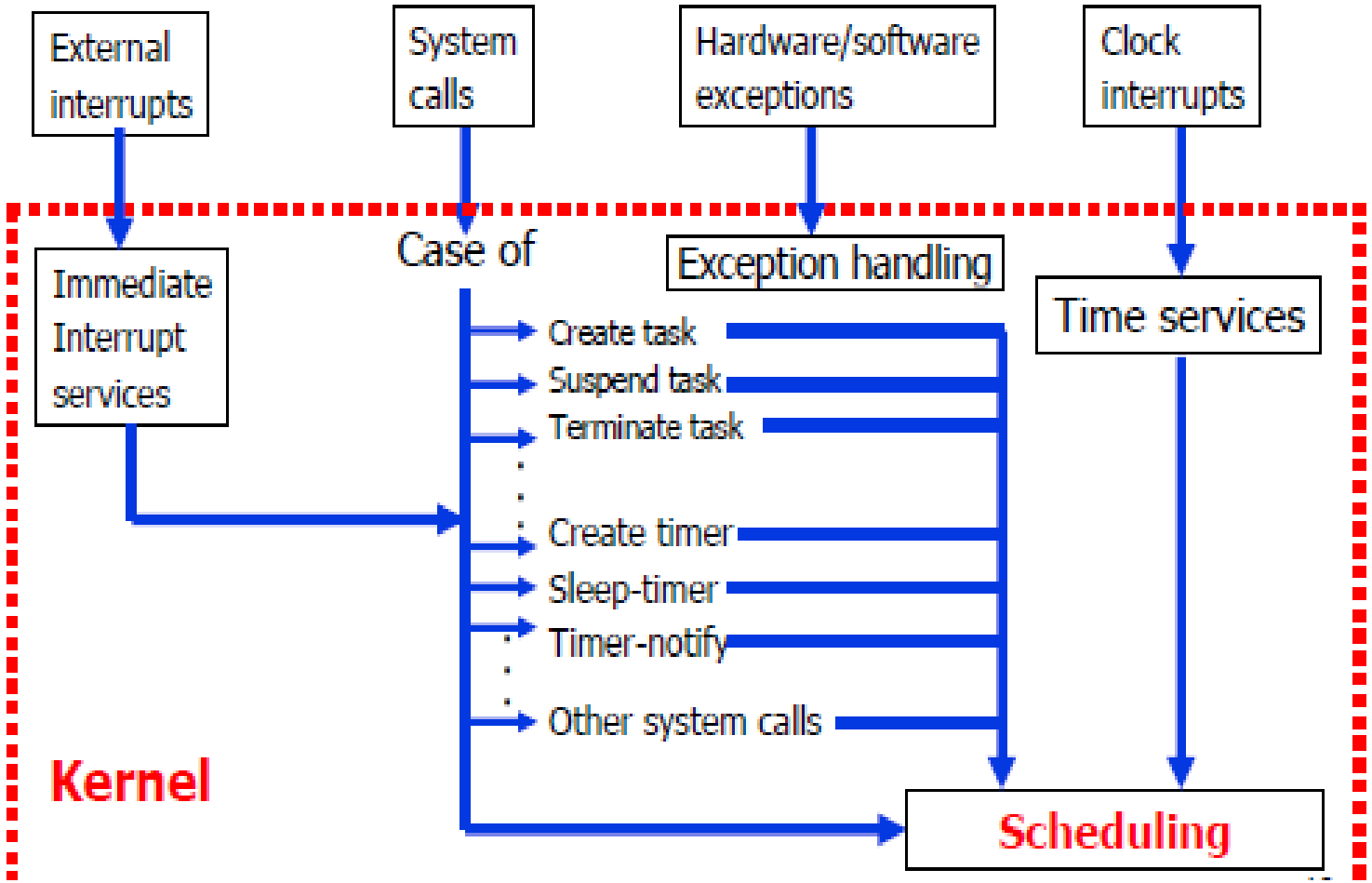
WHY OPERATING SYSTEMS FOR REAL-TIME APPLICATIONS

- complexity of applications expands beyond simple tasks eg: PDAs, cell phones, VCRs, industrial robot control.
- meet the system response requirements
- to be designed and expanded easily.
- It allows one to make better use of the system resources by providing valuable services such as semaphores, mailboxes, queues, time delays, time outs…etc.

Basic functions of RTOS kernel

- Task management
- Interrupt handling
- Memory management
  - no virtual memory for hard RT tasks
- Exception handling
- Task synchronization
  - Avoid priority inversion
- Task scheduling
- Time management

Micro-kernel architecture



Task

- Task = thread (lightweight process)
  - A sequential program in execution
  - It may communicate with other tasks
  - It may use system resources
- We may have timing constraints for tasks
- Task Classification by deadlines
  - Hard RT task must be computed within the given deadline (otherwise system failure), e.g ABS control
  - Soft RT task may be computed after the given deadline e.g.Multi-media, the web etc
  - On-time RT tasks (e.g alarm, robotics)

Task Classification

- Periodic tasks
  - arriving at fixed frequency, can be characterized by 3 parameters (C,D,T) where

    C = computing time

    D = deadline

    T = period (e.g. 20ms, or 50HZ)
  - Also called Time-driven tasks, their activations are generated by timers
- Non-Periodic or aperiodic tasks
  - all tasks that are not periodic, also known as Event-driven,
  - activations are generated by interrupts
- Sporadic tasks
  - aperiodic tasks with minimum interarrival time Tmin

Interrupt Handling

- **Types of interrupts**

  Asynchronous (or hardware interrupt) by hardware event (timer, network card …)

- Synchronous (or software interrupt, or a trap) by software instruction (swi in ARM, int in Intel 80x86), a divide by zero, a memory segmentation fault, etc.

Challenges in RTOS

- **Interrupt latency**
  - The time between the arrival of interrupt and the start of corresponding ISR.
  - Modern processors with multiple levels of caches and instruction pipelines that need to be reset before ISR can start might result in longer latency.
- **Interrupt enable/disable**
  - The capability to enable or disable (“mask”) interrupt individually.
- **Interrupt priority**
  - the ISR of a lower-priority interrupt is preempted by a higher-priority interrupt.

Memory Management/Protection

- Standard methods
  - Block-based, Paging, hardware mapping for protection
- No virtual memory for hard RT tasks
  - Lock all pages in main memory
- Many embedded RTS do not have memory protection – tasks may access any blocks
  - to achieve predictable timing
  - to avoid time overheads
- Most commercial RTOS provide memory protection as an option
  - Run into ”fail-safe” mode if an illegal access trap occurs
  - Useful for complex reconfigurable systems

Exception handling & Task Synchronization

- Exceptions e.g missing deadline, running out of memory, timeouts, deadlocks
  - Error at system level, e.g. deadlock
  - Error at task level, e.g. timeout
- Standard techniques:
  - System calls with error code
  - Watch dog
  - Fault-tolerance (later)
- Task Synchronization
  - **Synchronization primitives**
  a) **Semaphore**
  b) **Mutex**
  c) **Spinlock**
  d) **Read/write locks**
  e) **Barrier**

The Scheduler

The scheduler is at the heart of every kernel. A scheduler provides the algorithms needed to determine which task executes when.

Scheduling describes the following;

- schedulable entities,
- multitasking,
- context switching,
- dispatcher, and
- scheduling algorithms.

A schedulable entity is a kernel object that can compete for execution time on a system, based on a predefined scheduling algorithm.

Tasks and processes are all examples of schedulable entities found in most kernels.

A task is an independent thread of execution that contains a sequence of independently schedulable instructions.

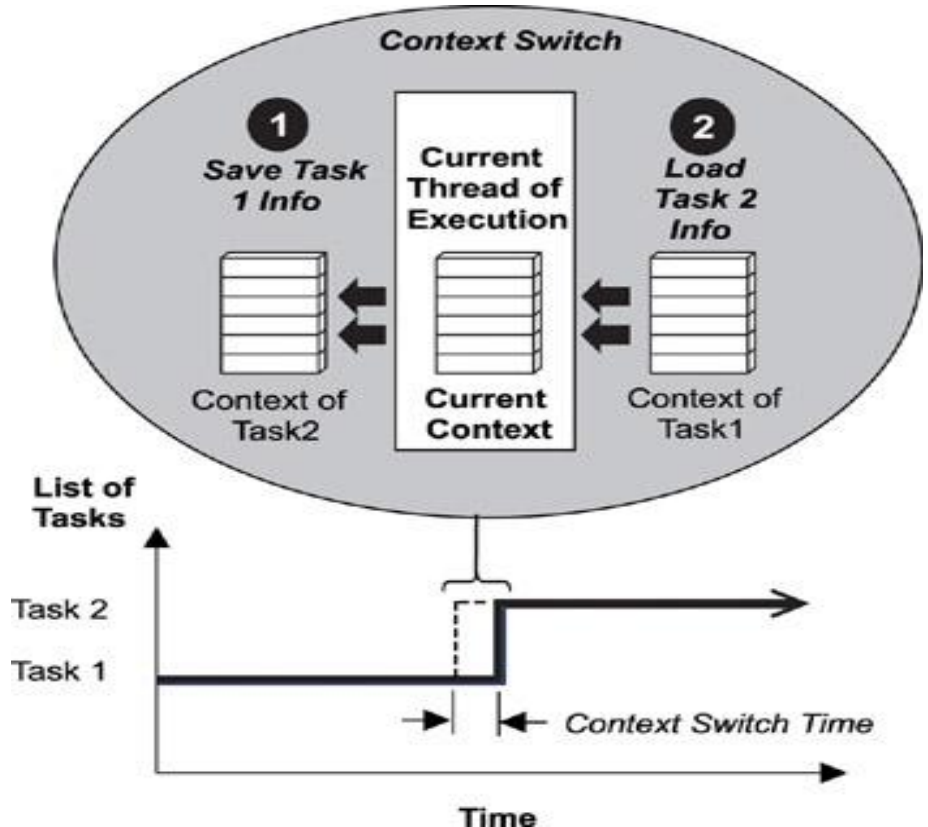
Processes are similar to tasks in that they can independently compete for CPU execution time.

Processes differ from tasks in that they provide better memory protection features, at the expense of performance and memory overhead.

Multitasking

Multitasking is the ability of the operating system to handle multiple activities within set deadlines. A real-time kernel might have multiple tasks that it has to schedule to run.

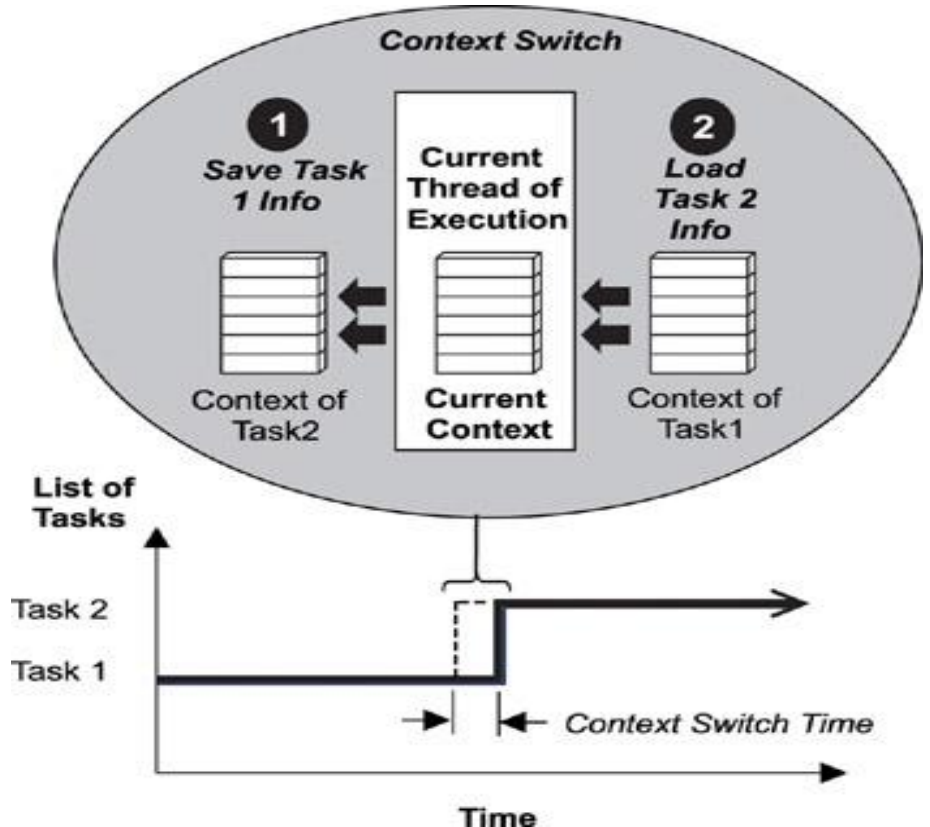
An important point to note here is that the tasks follow the kernel's scheduling algorithm, while interrupt service routines (ISR) are triggered to run because of hardware interrupts and their established priorities.
As the number of tasks to schedule increases, so do CPU performance requirements. This fact is due to increased switching between the contexts of the different threads of execution.`



The Context Switch

Each task has its own context, which is the state of the CPU registers required each time it is scheduled to run. A context switch occurs when the scheduler switches from one task to another.

Every time a new task is created, the kernel also creates and maintains an associated task control block (TCB). TCBs are system data structures that the kernel uses to maintain task specific information. TCBs contain everything a kernel needs to know about a particular task. When a task is running, its context is highly dynamic. This dynamic context is maintained in the TCB. When the task is not running, its context is frozen within the TCB, to be restored the next time the task runs.

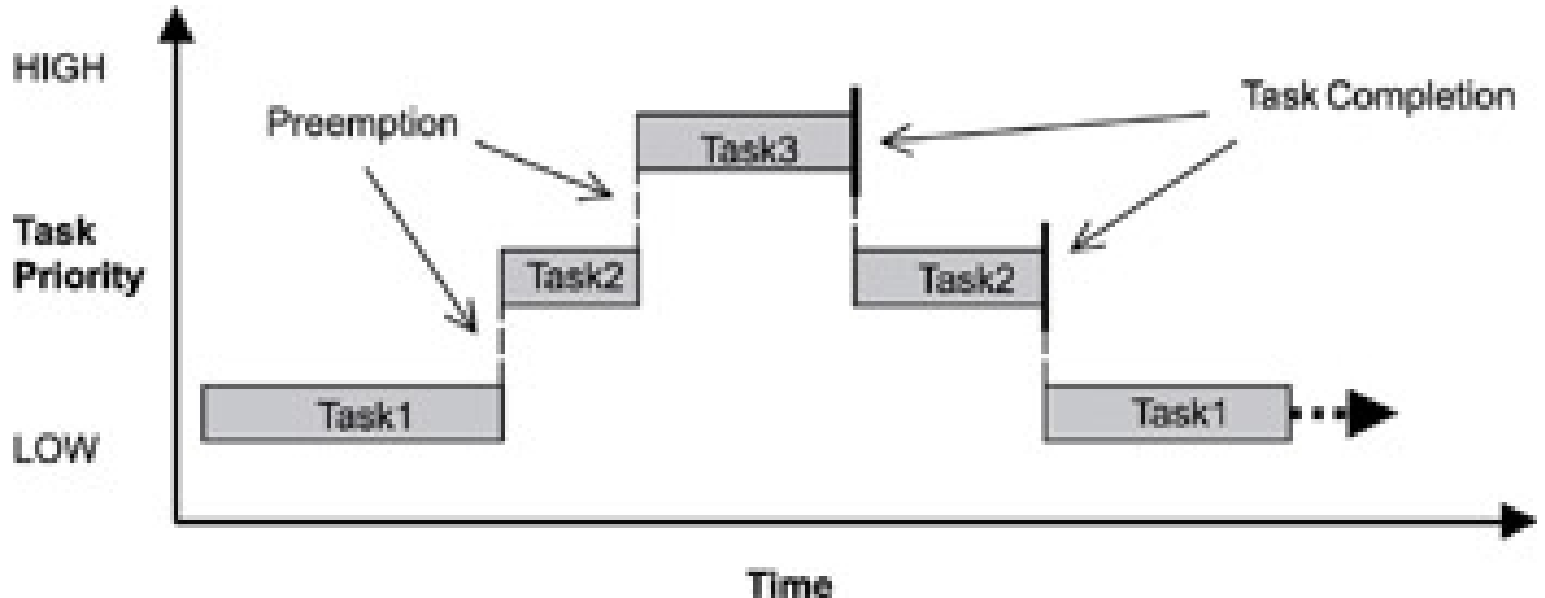


The Dispatcher

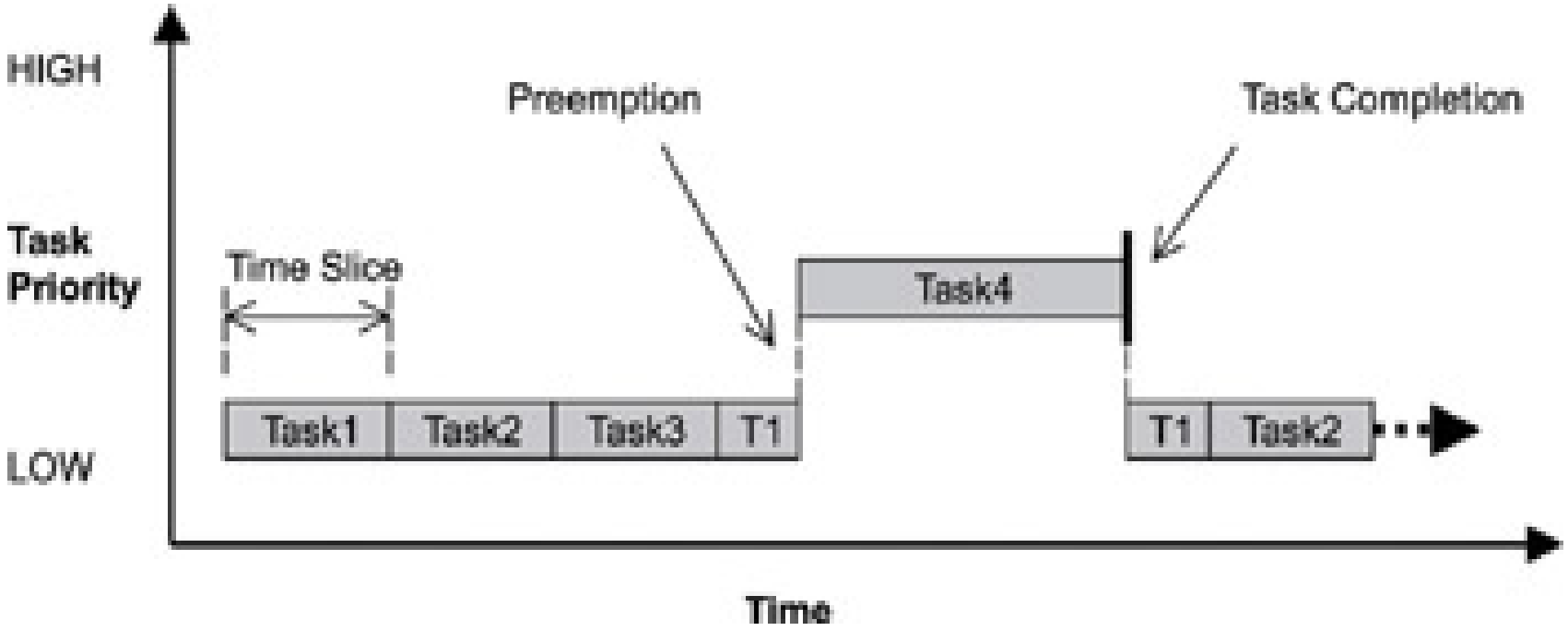
The dispatcher is the part of the scheduler that performs context switching and changes the flow of execution. At any time an RTOS is running, the flow of execution, also known as flow of control, is passing through one of three areas: through an application task, through an ISR, or through the kernel. When a task or ISR makes a system call, the flow of control passes to the kernel to execute one of the system routines provided by the kernel.

Scheduling Algorithms

preemptive priority-based scheduling
round-robin



Round-Robin Scheduling



POSIX

Stands for Portable Operating System Interface, a group of standards that defines an API to maintain compatibility Between different operating Systems.

Introduction

- Short for "Portable Operating System Interface for uni-X",
- POSIX is a set of standards codified by the IEEE and issued by ANSI and ISO.
- The goal of POSIX is to ease the task of cross-platform software development by establishing a set of guidelines for operating system vendors to follow.
- Ideally, a developer should have to write a program only once to run on all POSIX-compliant systems.
- Most modern commercial Unix implementations and many free ones are POSIX compliant.
- There are actually several different POSIX releases, but the most important are POSIX.1 and POSIX.2, which define system calls and command-line interface, respectively.
- The POSIX specifications describe an operating system that is similar to, but not necessarily the same as, Unix.
- Though POSIX is heavily based on the BSD and System V releases, non-Unix systems such as Microsoft's Windows NT and IBM's OpenEdition MVS are POSIX compliant.

What are the features that a RTOS has but a GPOS doesn't ?

1. Priority based preemptive scheduling mechanism
2. No or very short Critical sections which disables the preemption
3. Priority inversion avoidance
4. Bounded Interrupt latency
5. Bounded Scheduling latency , etc.

POSIX

- POSIX = **Portable Operating Systems Interface**
  - Standard enforced by IEEE
  - Implements a set of **automated conformance tests**

- Defines rules for a standard behaviour and interface, such as:
  - Standard C (Programming Language) operations
  - Multitasking
  - Error States
  - Command line & Commands

Overview of POSIX

- A major concern of POSIX:
  - Portability of applications across platforms at the source code-level.
- Now POSIX is widely accepted:
  - Even non-Unix operating systems are making themselves POSIX compliant.

POSIX

- POSIX is a family of standards or
- a collection of technical instruction manuals, created maintained and specified by IEEE computer society
- for maintaining ,compatibility between operating systems specifically UNIX operating system

Early POSIX

- As of 2009, POSIX documentation is divided in two parts:
- POSIX.1-2008: POSIX Base Definitions, System Interfaces, and Commands and Utilities (which include POSIX.1, extensions for POSIX.1, Real-time Services, Threads Interface, Real-time Extensions, Security Interface, Network File Access and Network Process-to-Process Communications, User Portability Extensions, Corrections and Extensions, Protection and Control Utilities and Batch System Utilities)
- POSIX Conformance Testing: A test suite for POSIX accompanies the standard: **PCTS** or the **POSIX Conformance Test Suite**.[6]

Open Software

- An open system is a vendor neutral environment, which allows users to intermix hardware, software, and networking solutions from different vendors.
- Open systems are based on open standards and are not copyrighted, saving users from expensive intellectual property right (IPR) law suits.
- The most important characteristics of open systems are: interoperability and portability.
- Interoperability means systems from multiple vendors can exchange information among each other.
- A system is portable if it can be moved from one environment to another without modifications.
- As part of the open system initiative, open software movement has become popular.

Advantages of open software

- It reduces cost of development and time to market a product.
- It helps increase the availability of add-on software packages.
- It enhances the ease of programming.
- It facilitates easy integration of separately developed modules.
- POSIX is an off-shoot of the open software movement.

Open Software standards can be divided into three categories:

- Open Source:
  - Provides portability at the source code level.
  - To run an application on a new platform would require only compilation and linking.
  - ANSI and POSIX are important open source standards.
- Open Object:
  - This standard provides portability of unlinked object modules across different platforms.
  - To run an application in a new environment, relinking of the object modules would be required.
- Open Binary:
  - This standard provides complete software portability across hardware platforms based on a common binary language structure.
  - An open binary product can be portable at the executable code level.
  - At the moment, no open binary standards.

ANSI and POSIX are important open source standards

- POSIX 1 : system interfaces and system call parameters
- POSIX 2 : shells and utilities
- POSIX 3 : test methods for verifying conformance to POSIX
- POSIX 4 : real-time extensions

Real-Time POSIX Standard

- POSIX.4 deals with real-time extensions to POSIX and is also known as POSIX-RT.
- The main requirements
  - Execution scheduling:
    - An operating system to be POSIX-RT compliant must provide support for real-time (static) priorities.
  - Performance requirements on system calls:
    - It specifies the worst case execution times required for most real-time operating services.

Real-Time POSIX Standard

- Priority levels:
  - The number of priority levels supported should be at least 32.
- Timers:
  - Periodic and one shot timers (also called watch dog timer) should be supported.
  - The system clock is called CLOCK REALTIME when the system supports real-time POSIX.

Cont.,

- Real-time files:
  - Real-time file system should be supported.
  - A real-time file system can pre-allocate storage for files and should be able to store file blocks contiguously on the disk.
  - This enables to have predictable delay in file access in virtual memory system.
- Memory locking:
  - Memory locking should be supported.
  - POSIX-RT defines the operating system services: mlockall() to lock all pages of a process, mlock() to lock a range of pages, and mlockpage() to lock only the current page.
  - The unlock services are munlockall(), munlock(), and munlockpage. Memory locking services have been introduced to support deterministic memory access.

Cont.,

- Multithreading support:
  - Real-time threading support is mandated. Real-time threads are schedulable entities of a real-time application that have individual timeliness constraints
  - may have collective timeliness constraints when belonging to a runnable set of threads.

Early POSIX:

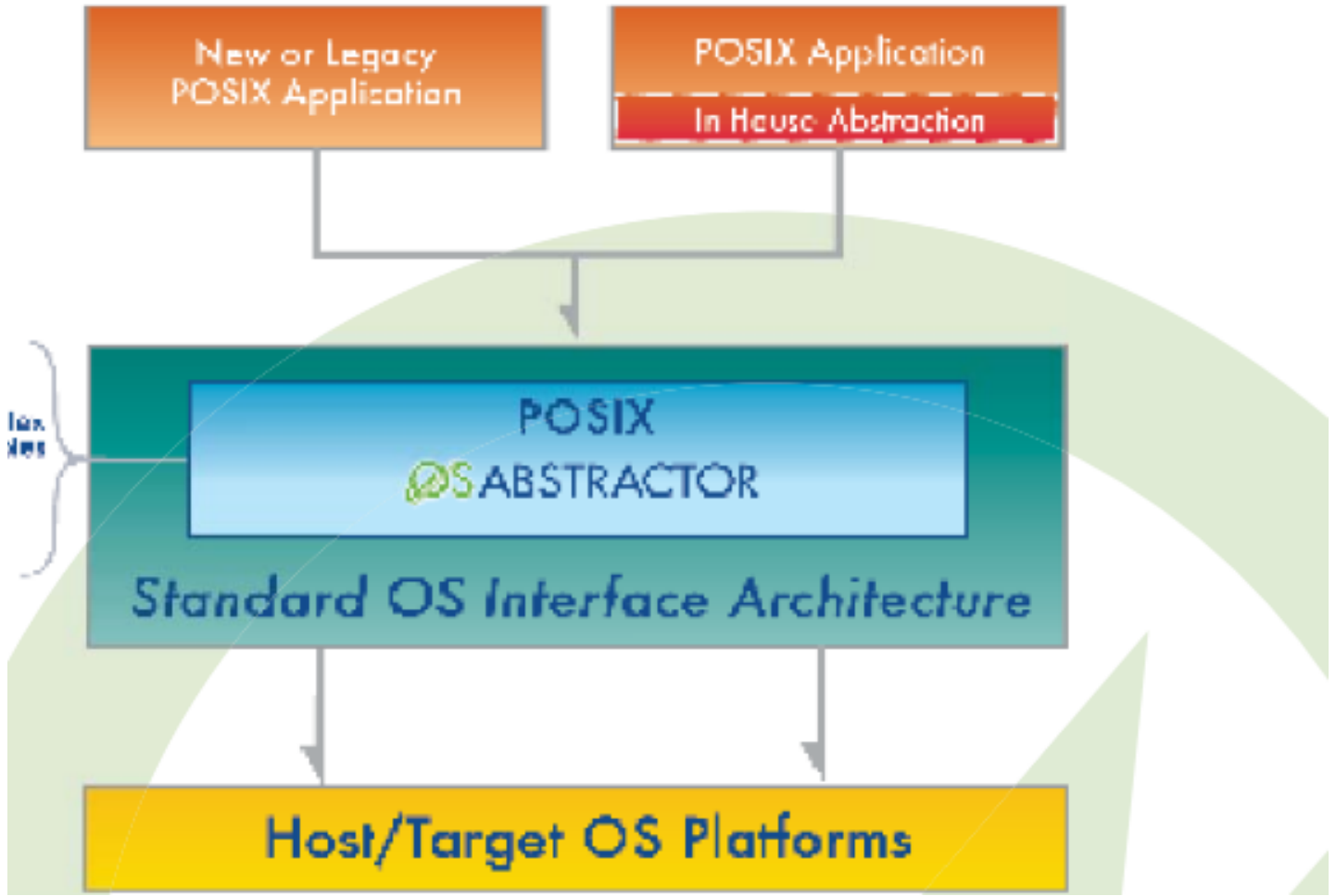
- **P**osix for Unix-like operating systems was first made up of one document for core pogramming interface, later it has grown to 19 document (POSIX.1, POSIX.2…etc.).
- However, nowadays most of Posix parts are combined into a single standard.

POSIX command line and scripting:

- **S**tandard command line and scripting interface were based on Korn shell. Also many user-level programs services, and utilities were also standardized along with program-level services (I/O, terminal, network).

Threading

- **P**osix defines a special threading library API which is supported by most modern operating system
- **POSIX Threads**, usually referred to as **Pthreads**.
- Implementations of the API are available on many Unix-like POSIX-conformant operating systems such as GNU/Linux, Mac OS X and Solaris. and Microsoft Windows implementations also exist.
- Pthreads defines a set of C programming language types, functions and constants. It is implemented with a library.


New or Legacy
POSIX Application
POSIX Application
In House Abstraction
POSIX
OSABSTRACTOR
Standard OS Interface Architecture
Host/Target OS Platforms

- Windows does not support the pthreads standard natively, therefore the Pthreads-w32 project seeks to provide a portable and open-source wrapper implementation. It can also be used to port Unix software (which use pthreads) with little or no modification to the Windows platform

File System Standard:

- **P**osix used 512 byte block instead of 1024 to reflect the default size of block on disk.
- they objected to this on the grounds that most people think in terms of 1024 byte (or 1 Kb) blocks. The environment variable POSIXLY_CORRECT was introduced to allow the user to force the standards-compliant behaviour.

POSIX-oriented operating systems:

- **O**perating systems can be classified as a POSIX-oriented depending upon the degree of compliance with the standards.
- Many Operating systems are conformed to be 100% compliant wit POSIX standards, such as Solaris,Unixware,IRIX.

POSIX for Windows

- Cygwin is a Unix-like environment for Windows.
- It contains a command-line interface and provides a native integration of Windows-based applications with software made for Unix-like environment.
- Microsoft POSIX subsystem, an optional Windows subsystem included in Windows NT-based operating systems up to Windows 2000.
- UWIN from AT&T Research implements

POSIX for DOS:

Partially POSIX compliant environments for DOS include:

- emx+gcc – largely POSIX compliant
- DJGPP – partially POSIX compliant
- DR-DOS multitasking core

C POSIX library:

- **C POSIX library** is a specification of a C standard library for POSIX systems. It was developed at the same time as the ANSI C standard. Some effort was made to make POSIX compatible with standard C; POSIX includes additional functions to those introduced in standard C.

C POSIX Library

- These are some header files of C POSIX Libraby:

<dirent.h>	Allows the opening and listing of directories.
<fcntl.h>	File opening, locking and other operations.
<grp.h>	User group information and control.
<pthread.h>	Defines an API for creating and manipulating POSIX threads.
<pwd.h>	passwd (user information) access and control.
<sys/ipc.h>	Inter-process communication (IPC).

Example POSIX APIs

- `pthread_create()`
- `pthread_exit()`
- `pthread_join()`
- `pthread_yield()`

Table I. List of POSIX Base Standards

POSIX.1	System Interface (basic reference standard)[a,b]
POSIX.2	Shell and Utilities[a]
POSIX.3	Methods for Testing Conformance to POSIX[a]
POSIX.4	Real-time Extensions
POSIX.4a	Threads Extensions
POSIX.4b	Additional Real-time Extensions
POSIX.6	Security Extensions
POSIX.7	System Administration
POSIX.8	Transparent File Access
POSIX.12	Protocol Independent Network Interfaces
POSIX.15	Batch Queuing Extensions
POSIX.17	Directory Services

[a] Approved IEEE standards
[b] Approved ISO/IEC standard

Table II. Additional POSIX Base Standards

P1224	Message Handling Services (X.400)
P1224.1	X.400 Application Portability Interface
P1238	Common OSI Application Portability Interface
P1238.1	FTAM OSI Application Portability Interface
P1201.1	Windowing Application Portability Interface
P1201.2	Recommended Practice on Driveability

Table III. List of POSIX Language Bindings

POSIX.5	Ada Bindings[a]
POSIX.9	Fortran 77 Bindings[a]
POSIX.16	C Language Bindings
POSIX.19	Fortran 90 Bindings
POSIX.20	Ada Bindings to Real-time Extensions
[a] Approved IEEE standards	

Table IV. List of POSIX Application Environment Standards

POSIX.0	Guide to POSIX Open System Environment
POSIX.10	Supercomputing Application Environment Profile
POSIX.11	Transaction Processing Application Environment Profile
POSIX.13	Real-time Application Environment Profiles
POSIX.14	Multiprocessing Application Environment Profile
POSIX.18	POSIX Platform Application Environment Profile

REAL-TIME EXTENSIONS

- Real-time Process Scheduling
  - SCHED_FIFO: This is a fixed-priority preemptive scheduling policy, in which processes with the same priority are treated in first-in-first-out (FIFO) order. At least 32 priority levels must be available for this policy.
  - SCHED_RR: This policy is similar to SCHED_FIFO, but uses a time-sliced (round robin) method to schedule processes with the same priorities. It also has 32 priority levels.
  - SCHED_OTHER: It is an implementation-defined scheduling policy.

- Virtual Memory Locking
  - Although virtual memory is not required by POSIX.1, it is common UNIX practice to provide this mechanism that has great benefits for non real-time software, but introduces large amounts of unpredictability in the timing response.
  - In order to bound memory access times, functions are defined in POSIX.4 to lock into physical memory either the whole address space of a process, or selected ranges of this address space.
  - These functions should be used for time-critical activities, and also for those activities with which they may synchronize. In this way, their response times can be made predictable.

THREADS EXTENSION

- Thread Management (creation , termination)
- Thread Synchronization
  - NO_PRIO_INHERIT(The priority of the thread does not depend on its ownership of mutexes)
  - PRIO_INHERIT(The thread owning a mutex inherits the priorities of the threads waiting to acquire that mutex)
  - PRIO_PROTECT(a thread locks a mutex it inherits the priority ceiling of the mutex, which is defined by the application as a mutex attribute).

- Thread Scheduling
  - Global Scheduling(The scheduler works only at the thread level, and the process scheduling parameters are ignored.)
  - Local Scheduling(the threads of the selected process compete among each other for the CPU)
  - Mixed Scheduling(first level, processes and global threads are scheduled; at the second level, local threads within the selected process are scheduled)